



Quantum Squeeziness: An Information Theoretical Metric for Quantum Software Testability

Avner Bensoussan

Department of Informatics, King's
College London
London, United Kingdom
avner.bensoussan@kcl.ac.uk

Hector Menendez

Department of Informatics, King's
College London
London, United Kingdom
hector.menendez@kcl.ac.uk

Mohammad Reza Mousavi

Department of Informatics, King's
College London
London, United Kingdom
mohammad.mousavi@kcl.ac.uk

Abstract

Testability—the degree to which faults can be revealed and diagnosed—has long been fundamental in software engineering but remains largely unexplored for quantum software. This work presents the first formalisation of testability for quantum programs, adapting classical concepts to quantum computation. We introduce *Quantum Squeeziness*, an information-theoretic metric inspired by classical squeeziness, to quantify how faults are masked and prevented from propagating to observable outcomes. We treat it as an inverse measure of testability: higher squeeziness implies stronger fault masking and reduced detectability.

We propose an efficient method to compute Quantum Squeeziness and validate it through large-scale quantum mutation analysis. Across more than 50k mutants, we observe a strong negative correlation between squeeziness and mutation score, showing that higher squeeziness predicts lower testability. We further introduce a technique that constrains where and how perturbations are injected, reducing cost while preserving accuracy. The computation is **15% faster than a standard error-propagation baseline**, while the optimised variant, *FastSqueeziness*, reduces mean runtime from 3200 s to 189 s (**16.9× speedup**) with **95% accuracy**.

By establishing a rigorous foundation of testability for quantum software, this work opens new directions at the intersection of software engineering, quantum information theory, and testing. Beyond testing and debugging, it also supports applications in quantum error mitigation and correction, enabling principled assessment and improvement of program robustness.

CCS Concepts

• Software and its engineering → Designing software.

Keywords

Quantum software testing, information theory, quantum testability

ACM Reference Format:

Avner Bensoussan, Hector Menendez, and Mohammad Reza Mousavi. 2026. Quantum Squeeziness: An Information Theoretical Metric for Quantum Software Testability. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26)*, October 12–16, 2026, Munich, Germany. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3832783.3834341>



This work is licensed under a Creative Commons Attribution 4.0 International License. ASE '26, Munich, Germany

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
<https://doi.org/10.1145/3832783.3834341>

1 Introduction

As quantum computing moves toward practical applicability, the correctness and reliability of quantum programs have become increasingly critical [16, 38, 41]. Despite substantial progress in quantum algorithms, compilers, and hardware, the fundamental software engineering concept of testability—the extent to which faults can be revealed and diagnosed—remains largely unexplored in the quantum domain. Quantum software must not only deliver correct output, but also remain robust and predictable in the face of hardware noise, decoherence, and subtle quantum-specific faults such as phase flips or unintended entanglement. Without sufficiently high testability, faults may remain hidden, undermining confidence in both correctness and long-term reliability. Indeed, in classical software testing, testability is widely recognised as foundational to fault detection, maintainability, and software quality [6, 17]. In the quantum domain, classical notions of testability have no direct translation. Superposition, entanglement, and probabilistic measurement fundamentally reshape program behaviour and observability, while the potential exponential growth of state space means that small errors can propagate in counterintuitive ways. Observable outcomes may therefore not sensitively reflect internal state discrepancies, motivating the need for quantum-aware testability frameworks rather than naïvely re-using classical approaches [15, 43].

To address this gap, we propose a quantum-native formalism for testability, grounded in information-theoretic foundations and inspired by classical testability metrics. Central to our formulation is the concept of squeeziness [9]: the degree to which information about a fault in a quantum state is lost or “squeezed” through the circuit, making it difficult to observe in standard output statistics. Our definition is based on comparing quantum states, and we start with measuring that using observable outputs. By modelling how faults propagate through quantum states and how they influence (or fail to influence) the final state, our approach provides a principled means for assessing testability with respect to failure and error propagation. This allows us to quantify the *testability* of a quantum program, i.e., how reliably faults manifest as observable deviations rather than being masked by quantum effects. We provide a more efficient technique to confine the scope in which faults are injected and evaluated for testability. Further improvements may be achieved by incorporating measurement-based methods such as classical shadows [4, 23].

Beyond its conceptual contribution, Quantum Squeeziness is designed for automation. It can support mutant selection, test-effort prioritisation, oracle-strength selection, and fitness-guided fuzzing or circuit generation. Because it requires no human judgement

and operates directly on circuit behaviour, it provides an operational notion of testability for quantum software under probabilistic execution and noise.

Research Questions. To bridge the gap between established software testing principles and the emerging needs of quantum computing, this study examines how the concept of testability can be meaningfully adapted to quantum programs. Building on classical foundations while accounting for the unique behaviours of quantum systems, we investigate how faults propagate through quantum states, how such propagation can be quantified, and what these metrics imply in terms of test effort and the effectiveness of fault detection. Guided by this goal, we structure our inquiry around the following research questions.

- **RQ1. Definitions.** How can testability and squeeziness be formally defined for quantum software systems, and what validation requirements must such definitions satisfy?
This question investigates classical notions of testability and squeeziness and their foundations in information theory, to extend these concepts to quantum software. It further aims to define the necessary validation criteria for quantum software testability.
- **RQ2. Metrics.** How can testability and squeeziness be quantitatively measured in quantum software?
This question develops metrics for both *quantum software testability* and *Quantum Squeeziness*, evaluates their interrelation, and compares their efficiency and effectiveness. It also investigates whether Quantum Squeeziness exhibits an inverse correlation with testability, as observed in classical software [9].
- **RQ3. Implications for Testing.** How does squeeziness relate to testing?
This question investigates the practical applications of Quantum Squeeziness, particularly its impact on the effectiveness of testing oracles through its correlation with mutation analysis, and its potential to guide the selection and prioritisation of more testable quantum circuits.

This study offers three main novel contributions.

- We **transfer the notion of testability to a Quantum context** and present a **first metric** to assess it.
- We propose a **quantum counterpart of squeeziness**, an information-theoretic measure of **software testability**, and evaluate its **correlation with quantum testability**.
- We **validate our metrics over mutation analysis** to ensure its relevance to quantum software testing.

Structure of the paper. Section 2 introduces quantum computing, and key concepts to understand testability and its use cases in Software Engineering. In Section 3, we review the related work, both in classical and quantum software engineering. Building on the two previous sections, Section 4 provides an intuition as well as a set of requirements for quantum testability. Section 5 describes the methodology of this study, and the experimental setup we followed. In Section 6, we present our results before discussing them in Section 7. We state threats to the validity of our study in Section 8, suggest some avenues for future work in Section 9 and conclude in Section 10.

2 Background

In this section, we briefly introduce quantum computing and key elements to understand testability and fault masking.

2.1 Quantum Computing

Quantum computing (QC) was conceptualised to simulate quantum mechanics with computers "*built of quantum mechanical elements which obey quantum mechanical laws*" [13]. It was later found to have several potential applications and could offer significant speed-up over its classical counterpart. In 1994, Shor's proposal of a polynomial-time algorithm for prime factorisation and discrete logarithms on a Quantum Computer [51] raised enormous interest due to the threat it could constitute to modern RSA cryptosystems. Soon after, Grover proposed a fast database search on quantum computers [19] that promised quadratic speed-up over the best classical algorithm. The resulting potential speed-up is often referred to as "quantum supremacy" [12]. However, demonstrating quantum supremacy on real hardware remains a long-standing challenge. In the near term, the focus is on achieving quantum advantage—showing that quantum devices can outperform classical computers on specific, practical tasks. As Preskill notes [48], this marks the first major milestone for quantum computing.

2.2 Quantum Circuits

A quantum circuit typically begins with the initialisation of qubits and ends with their measurement, where the information is translated into classical bits. The purpose of a quantum circuit is to harness the principles of quantum mechanics, such as superposition and entanglement, to perform computational tasks that would be significantly more efficient than classical computers. A qubit in superposition is represented as a linear combination of the computational basis states,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$. Unlike a classical bit, which must occupy exactly one state at any given time, a qubit exists in a coherent combination of both basis states prior to measurement. The squared magnitudes $|\alpha|^2$ and $|\beta|^2$ correspond to the probabilities of obtaining $|0\rangle$ or $|1\rangle$, respectively, upon measurement. Quantum circuits exploit this property to process a multitude of possibilities simultaneously, enabling quantum parallelism [42]. Quantum circuits can create entangled states where the properties of one qubit are dependent on the properties of another, even if they are physically separated. This entanglement allows for new information processing using quantum parallelism [42].

Figure 1 represents a simple four-qubit quantum circuit that is designed to produce an equally balanced distribution of all outputs. Input variables $q[0]$ to $q[3]$ represent the 4 input qubits. The line represents time. The square and round elements are quantum logic gates, and the final boxes elements with a needle inside represent measurements, where the qubit is observed and its state is extracted into classical bits. The gate depth of a quantum circuit measures its computational complexity as the number of sequential layers of quantum gates required from input to measurement [42]. Figure 1 represents a circuit of depth 4.

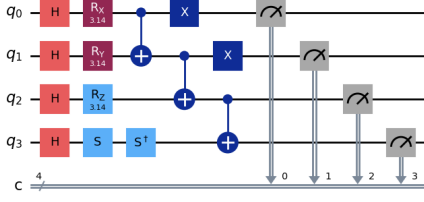


Figure 1: Simple quantum circuit creating an equally balanced distribution of all outputs.

State-level representation and fault abstraction. Any quantum circuit C acting on an initial state ρ_0 prepares a quantum state $\rho = U\rho_0U^\dagger$, where U is the unitary implemented by the circuit; for pure states, $\rho = |\psi\rangle\langle\psi|$. More generally, any physically realizable quantum process is described by a completely positive trace-preserving (CPTP) map, i.e., a quantum channel acting on density operators [42, 55]. By the Church–Turing–Deutsch principle [10], any implementable operation admits such a representation. Modelling perturbations at the state (or channel) level therefore captures the most general semantic effect of faults, independent of their syntactic origin in the circuit, whether developer-introduced (e.g., incorrect gates or parameters) or hardware-induced, and thus forms the basis of our approach in this paper.

Similarity metrics. A quantum state can be represented either as a state vector or, more generally, as a *density matrix* ρ , which captures both pure states and statistical mixtures. Comparing quantum states therefore reduces to comparing their corresponding matrices. Two widely used metrics for this purpose are **quantum fidelity** and the **quantum Hellinger distance**, both of which quantify how similar two quantum states are [42].

The *quantum fidelity* between two density matrices ρ and σ is defined as [30]:

$$F(\rho, \sigma) = \left(\text{Tr} \sqrt{\sqrt{\rho} \sigma \sqrt{\rho}} \right)^2, \quad (1)$$

where ρ and σ are density operators and Tr denotes the trace. It measures how closely the two states resemble one another, with $F(\rho, \sigma) = 1$ if and only if $\rho = \sigma$, and values closer to 0 indicating increasingly dissimilar states.

Complementarily, the *quantum Hellinger distance* [20] provides a notion of distance between states:

$$H_Q(\rho, \sigma) = \frac{1}{\sqrt{2}} \|\sqrt{\rho} - \sqrt{\sigma}\|_F, \quad (2)$$

where $\|A\|_F = \sqrt{\text{Tr}(A^\dagger A)}$ denotes the Frobenius norm. While fidelity measures similarity, the Hellinger distance quantifies dissimilarity, taking the value 0 when the states are identical and increasing as they diverge. The two metrics are directly related by

$$H_Q(\rho, \sigma) = \sqrt{1 - \sqrt{F(\rho, \sigma)}}, \quad (3)$$

highlighting their shared geometric interpretation in the space of quantum states [30, 42].

2.3 Testability in Software Engineering

Testability. refers to the degree to which a system or component supports testing efforts, influencing how efficiently one can identify faults and verify correct functionality. Several definitions of testability exist in the literature. Both the IEEE and ISO standards relate testability to the ease of establishing the test criterion, and the performance of test to assess whether the criteria were met [26, 27]. Testability is associated with two key notions according to Binder [6]: controllability and observability. One must be able to control the input and observe the output of a system to properly test it. Voas et al. [52] focus on the sensitivity to faults. They relate testability to a notion of information loss between the input and output domains by looking at software dynamics, not just syntax. Finally, Briand et al. [7] define testability as the extent to which a module exposes information that facilitates the automatic generation of test cases. We adopt the approach of Voas et al. [52] and adapt it to quantum software.

Fault Propagation. A **fault** is a defect or flaw in a system component that can lead to incorrect behavior or outputs. Faults can arise due to design flaws, manufacturing defects, environmental factors, or operational developer mistakes.

Failed Error Propagation (FEP) refers to situations where an error, originating from a fault, does not propagate to the output, making it undetectable by typical testing methods [2]. This lack of error propagation can lead to "silent" errors, where faults go unnoticed by error detection mechanisms, thus impacting the reliability assessment of the system. FEP directly undermines observability: if a fault does not propagate into an error, then the observable outputs appear correct even in the presence of a latent fault. The study of error propagation has been a widely recognised technique to assess the testability of a SUT since the early 90s [40]. FEP analysis through fault injection studies is therefore a common way of assessing the testability of a system [35].

Squeeziness. is defined as a measure of the information (entropy) loss during program execution. For a deterministic program that maps inputs to outputs, let I denote the random variable over the input domain and O the induced random variable over the output domain. Then, the squeeziness is given by:

$$Sq(I, O) = H(I) - H(O) \quad (4)$$

where $H(X)$ is the Shannon entropy of the random variable X , defined as follows [50]:

$$H(X) = - \sum_{x \in X} p(x) \log p(x), \quad (5)$$

where X is a discrete random variable and $p(x)$ is the probability of each outcome x . This measure quantifies the uncertainty or information content inherent to the probability distribution of X .

The extremes of squeeziness are illustrated by the uniform and Dirac cases. If the input I is uniformly distributed over n values, then $H(I) = \log n$. If the program maps every input to a single output value (a Dirac distribution), then $H(O) = 0$, and therefore $Sq(I, O) = \log n$, which is the maximum possible information loss. Conversely, if I is itself a Dirac distribution, then $H(I) = 0$ and, since the program is deterministic, $H(O) = 0$, giving $Sq(I, O) = 0$, the minimum possible information loss. For intuition, consider a

simple classical function: if it always returns the same output, all information about the input is lost (high squeeziness), while if each input maps uniquely to an output, the input information is fully preserved (low squeeziness).

Squeeziness was proven to be efficient in several testing contexts, noticeably to assess Failed Error Propagation [25], or to detect fault-masking [46]. Androutsopoulos et al. [2] studied the relationship between Conditional Entropy and Failed Error Propagation in software testing, highlighting once more the close correlation between information theory and testability. Although quantum information theory is a very active field of research [3], to our knowledge, it has not yet been utilised in the context of quantum software testability.

3 Related Work

In this section we first present the state-of-the-art in quantum software testing, introduce how information theory is used to enhance classical software testability, and then describe how testability serves in establishing software design guidance.

3.1 Testing Quantum Programs

Quantum software testing is challenging, with multiple studies highlighting the critical need for specialised testing and debugging tools for quantum programs [18, 39, 41, 44, 45]. Several testing approaches have been explored, including property-based testing [22, 47], mutation-based [14, 37, 54], statistical assertion [24, 31, 34], search-based [53] and metamorphic testing [1] techniques. Complementing these efforts, researchers have developed quantum bug benchmarks [8, 36, 60] and identified specific bug patterns, notably in Qiskit programs [59]. MQTBench suite [49] provides a quantum circuit benchmark to support quantum software research, which we used in this study. Together, these efforts highlight the progress and gaps in quantum software testing, motivating our approach, which focuses on quantifying program testability and providing a principled method to identify circuits where faults are most likely to manifest.

3.2 Fault Masking in Logic and Reversible Circuits

Fault masking prevents faults from propagating to observable outputs, typically using redundancy or error-correcting schemes such as triple modular redundancy (TMR) or error-correcting codes (ECC) in classical circuits [11, 28, 29]. Zamani et al. [58] extended these ideas to reversible circuits, linking them to quantum computing and quantum error correction. These techniques provide intuition for quantum circuits, where superposition and interference can similarly mask faults, reducing their observability. Our work builds on this insight by quantifying fault propagation and masking to provide a principled measure of quantum program testability.

3.3 Information Theory for Software Testability

Testability of classical software has been an active area of research for decades. In 1995, Voas et al. [52] linked testability to a notion of implicit information loss. They introduced a Domain to Range Ratio (DRR) that assesses the information loss between the input and output distribution, and allows for assessment of the likelihood

of fault propagation in software. Later, Clark et al. [9] introduced an information-theoretic measure of testability, Squeeziness, that performs more efficiently than DRR.

3.4 Testability for Software Design Guidance

Testability is not only a valuable asset to guide software testing, but its analysis can also lead to general guidelines to develop safer and more stable software. Binder [6] suggested that testability should be considered before development, and motivate design choices to maximise controllability and observability. As systems' complexity increases, testing schemes become more expensive, and designing for optimal testability becomes crucial. Woodward et al. [56] evaluate the relationships between testability, Fault size, and Domain-to-Range Ratio, suggesting that the balance between these three notions should be considered when designing software. Voas et al. [52] conclude their work with '*... we are convinced that this inherent software characteristic [testability] will become an important factor to consider during software development and assessment.*' Similarly, we believe defining testability in a quantum software context is a crucial step towards not only better quantum testing techniques but also towards guidance to more stable quantum software design.

3.5 Positioning our Study

The related work in Table 1 highlights that existing quantum software engineering approaches typically address only isolated aspects of our problem space. Prior quantum-focused tools such as MorphQ [45] and Muskit [37] provide algorithm-level measurements and practical tool support, whereas statistical assertion frameworks [24, 34] and bug benchmarks [8, 60] cover narrower evaluation needs without formal definitions or information-theoretic foundations. In contrast, classical software-engineering measures—such as DRR [56], squeeziness [9], and entropy-based analyses [2, 25, 46]—offer formal definitions and leverage information theory, but operate exclusively in classical domains and lack quantum applicability. Our work uniquely integrates all four pillars: a formal definition of testability, its efficient algorithmic measurement, quantum-native information-theoretic grounding, and an implemented tool, thereby filling a gap addressed by neither prior quantum nor classical approaches.

4 Quantum Testability: Intuition and Requirements

While classical testability is well defined, quantum software lacks a native, formal notion. Building on the concepts introduced in Sections 2 and 3, we outline the intuition and requirements for *quantum testability*.

4.1 Intuition

A *testable quantum circuit* should allow faults to propagate to observable outputs, directly reflecting classical notions of *observability* and *sensitivity to fault* [6, 52]. Conversely, if a perturbation does not affect the output, the circuit exhibits a quantum analogue of *Failed Error Propagation (FEP)* [2], thus undermining testability.

However, unlike classical programs, internal quantum states cannot be inspected. As a result, testability must be inferred from

Table 1: Positioning of our work with respect to related research.

Work	Application Domain	Formal Definition	Automated Measurement	Information-Theoretic Foundation	Software Implementation
Quantum-Focused Testing Techniques [37, 45]	Q	–	✓	–	✓
Statistical Assertions [24, 34]	Q	–	✓	–	–
Bug Benchmarks [8, 60]	Q	–	–	–	✓
QASMBench [33]	Q	–	–	–	✓
DRR [52]	C	✓	✓	–	–
Squeeziness [9]	C	✓	✓	✓	–
Entropy-Based [2, 25, 46]	C	✓	✓	✓	–
Our Work	Q	✓	✓	✓	✓

how differences in the quantum state manifest at the output - here the final state vector, consistent with information-loss-based views of testability [25, 46].

Scalable and state-grounded assessment of testability. Testability is inherently a property of quantum states, but directly accessing state vectors is infeasible for large circuits. In this work, we define and analyze testability in a fully controlled, state-vector setting, where perturbations can be precisely applied and their effects directly observed. This controlled approach establishes the fundamental notions and formal properties of testability [2, 6, 50]. At the same time, our methodology is designed to be scalable: practical proxies based on weak or mid-circuit measurements and classical-shadow techniques [4, 23] can extend testability assessment to larger circuits, including those relevant for the fault-tolerant quantum computing era, without requiring full state access.

4.2 Requirements on a Quantum Testability Spectrum

We posit a natural *testability spectrum*, defined by how much an injected perturbation is preserved in the output – a direct analogue of classical FEP and *squeeziness*-based reasoning (Eq. 4) [2, 25].

At one extreme, *Constant Circuits* erase input distinctions, producing maximal information loss and strong fault masking. At the other, *Identity Circuits* preserve all deviations, maximising sensitivity and observability [52]. Between these lie general quantum programs that partially suppress or propagate perturbations.

We therefore set the following requirements:

- (1) Weak propagation of perturbations implies low testability.
- (2) Strong propagation of perturbations implies high testability.
- (3) Quantum testability is fundamentally tied to information retention.

In Section 5, this intuition is operationalised using state-level perturbation injection and systematic comparison of the resulting output states.

4.3 Formal Definitions

Let C be a quantum circuit acting on n qubits, and let $\mathcal{E}_C$ denote the induced completely positive trace-preserving (CPTP) map [?]. Given an initial state ρ_0 , the nominal output of the circuit is

$$\rho = \mathcal{E}_C(\rho_0).$$

To analyze fault propagation, we introduce a *circuit cut* at layer k , decomposing the circuit into a *prefix* channel $\mathcal{E}_{C_{0 \rightarrow k}}$ and a *suffix* channel $\mathcal{E}_{C_{k \rightarrow \text{end}}}$. The intermediate state at the cut is

$$\rho_k = \mathcal{E}_{C_{0 \rightarrow k}}(\rho_0),$$

and the final output can equivalently be written as

$$\rho = \mathcal{E}_{C_{k \rightarrow \text{end}}}(\rho_k).$$

A controlled perturbation of magnitude δ applied at the cut is modeled as a CPTP map $\mathcal{F}_{k,\delta}$, producing the perturbed output

$$\rho^{(k,\delta)} = \mathcal{E}_{C_{k \rightarrow \text{end}}}(\mathcal{F}_{k,\delta}(\rho_k)).$$

Let $d(\cdot, \cdot)$ be a quantum distance measure (e.g., Hellinger distance). The notion of quantum testability is formally defined as follows:

Definition 1 (Quantum Testability). The quantum testability of C is the expected observable deviation under controlled perturbations:

$$T(C) = \mathbb{E}_{k,\delta} \left[d\left(\rho, \rho^{(k,\delta)}\right) \right].$$

High $T(C)$ indicates strong fault propagation, while low $T(C)$ indicates fault masking.

Here, $\mathbb{E}_{k,\delta}$ averages over cut locations k and perturbation magnitudes δ . Quantum Squeeziness (Definition 2) measures how much information about injected faults is lost, capturing the circuit's tendency to mask or reveal perturbations.

Definition 2 (Quantum Squeeziness). The circuit can also be viewed as an information channel mapping perturbation magnitudes X to observable deviations Y . Let $H(\cdot)$ denote Shannon entropy. Quantum Squeeziness is defined as

$$QS(C) = H(X) - H(Y) = H(X | Y),$$

quantifying the information about injected perturbations that is lost during computation.

4.4 Example: Quantum Squeeziness in Simple Circuits

To illustrate Quantum Testability and Squeeziness, we consider simple single-qubit circuits and analyze how perturbations propagate to the output.

Identity Circuit. Consider

$$|0\rangle \rightarrow I \rightarrow \text{Measure},$$

with a perturbation $R_X(\delta)$ applied (representing the error in quantum state due to a fault):

$$|0\rangle \rightarrow R_X(\delta) \rightarrow I \rightarrow \text{Measure}.$$

The perturbation directly changes the measurement probabilities, so the disturbance is observable and the circuit exhibits *low Quantum Squeeziness*. Formally, taking the cut before measurement, $\rho_k = |0\rangle\langle 0|$ and $\rho^{(k,\delta)} = R_X(\delta)\rho_k R_X^\dagger(\delta)$. By Definition 1,

$T(C) = d(\rho, \rho^{(k, \delta)}) \propto \delta$, and by Definition 2, $QS(C) \approx 0$, confirming high testability and low squeeziness. Such circuits are easier to test because faults propagate clearly to the measurement.

Perturbation Masking Circuit. Now consider a semantically equivalent circuit, where H denotes the Hadamard Gate:

$|0\rangle \rightarrow H \rightarrow H \rightarrow \text{Measure}.$

Since $HH = I$, the circuit behaves as an identity transformation in the fault-free case. If the same perturbation is inserted between the two gates,

$$|0\rangle \rightarrow H \rightarrow R_x(\delta) \rightarrow H \rightarrow \text{Measure},$$

the first Hadamard spreads the state into a superposition. When the perturbation acts in this basis, its effect becomes distributed across amplitudes, and the second Hadamard recombines them through interference. This recombination tends to reduce the observable impact of the perturbation, partially masking the fault.

Less information about the disturbance therefore reaches the output, meaning the circuit exhibits *higher Quantum Squeeziness* and reduced fault observability.

Implications for Testing. Quantum Squeeziness therefore captures how much information about injected disturbances survives until measurement: circuits with low squeeziness expose faults clearly, while circuits with high squeeziness compress or mask their observable effects, making testing more challenging.

5 Methodology

This section describes the methodology employed to answer our research questions.

5.1 Research Questions: Methodological Scope and Validation

We address three research questions corresponding to *definitions*, *metrics*, and *implications for testing*, detailing how each is answered and validated.

RQ1 – Definitions. How can quantum software testability and Quantum Squeeziness be formally defined? We formalise these notions by extending classical testability and information-theoretic principles to the quantum setting. Validation is conceptual, ensuring alignment with the intuition and testability spectrum, and by examining extreme cases such as Constant and Identity circuits.

***RQ2 – Metrics.* How can quantum software testability and Quantum Squeeziness be quantitatively measured?** We introduce operational metrics based on systematic fault injection, output deviation, and information-theoretic analysis. Validation is empirical using a representative circuit benchmark, checking consistency with the testability spectrum and correlations between testability and squeeziness.

RQ3 – Implications for Testing. **How does Quantum Squeeziness relate to practical fault detection in quantum programs?** We assess this by correlating squeeziness with mutation scores across a large set of mutants, demonstrating predictive power for

fault detectability and guiding circuit selection and test prioritisation.

5.2 Conceptual Foundations: Testability and Squeeziness

Body of Knowledge and Systematic Mapping of Classical Software Testability. To address RQ1, we relied on the holistic study of software testability by Garousi et al. [17], which systematically examined 208 papers published between 1982 and 2017 following rigorous inclusion criteria. Their work provides a comprehensive overview of the classical software testability body of knowledge but does not explicitly link testability to information theory.

To bridge this gap, we incorporated the influential work of Clark et al. on squeeziness [9] and expanded this literature set using a *snowballing approach* [21]. In this context, snowballing refers to iteratively extending the set of primary studies through both backward and forward reference analysis of key papers. From these sources, we extracted the core notions relevant to testability and analysed their applicability and potential challenges within the context of quantum computing.

We first attempted a naive translation of the classical Domain and Range notions [52]: Domain would be defined as all possible classical initialisation of the qubits, and Range as all possible classical output distributions. Squeeziness was computed as the difference of entropy between the domain and the range. This first attempt, referred as ShanonSqueeziness in the replication package, led to results violating the requirements set in Section 4.2.

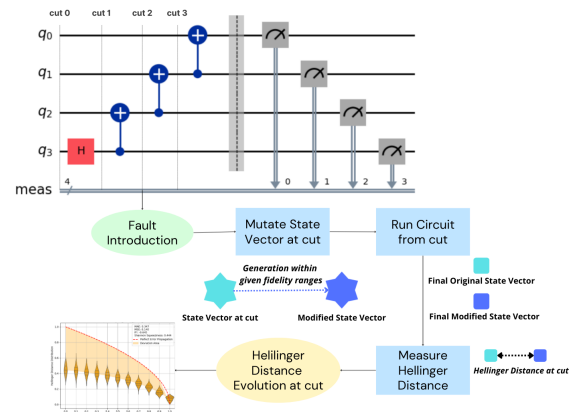


Figure 2: Pipeline for establishing the baseline of quantum circuit testability.

Our formalisation is grounded in fundamental principles of quantum software behaviour, including sensitivity to perturbations and information preservation under unitary evolution.

5.3 Metrics for Quantifying Testability and Squeeziness

To operationalise our definitions, we developed quantitative metrics for both *quantum software testability* and *Quantum Squeeziness*, enabling systematic evaluation of circuits across diverse benchmarks.

Representative Circuit Batch (Validation Dataset). To illustrate the spectrum of testability, we selected a diverse batch of quantum circuits with known behaviours. These circuits serve as a validation dataset to assess whether the proposed metrics align with the intuitive notion of testability. We expect error propagation to increase from the least to the most testable circuit:

- **Constant Circuit:** 2 qubits; operations applied solely to the first qubit, second qubit unchanged. Qubits are swapped at the end, and only qubit 0 is measured. Disturbances are injected solely into the first qubit, leading to minimal error propagation.
- **Equally Balanced Output Circuit:** Designed to systematically output all possible outcomes with equal probability.
- **Quantum Fourier Transform (QFT) Entangled Circuit:** Produces several dominant outputs due to entanglement.
- **Amplitude Estimation Circuit:** Results in multiple dominant outputs reflecting probabilistic estimation.
- **GHZ State Circuit:** Equally entangles all qubits, producing only two equally probable states.
- **Grover Algorithm Circuit:** Oracle-based algorithm producing one dominant output through amplitude amplification.
- **Identity Implementations:** Three logically equivalent circuits producing identical final states; expected to be most sensitive to error propagation.

This circuit batch is intentionally curated to span a wide range of expected testability behaviours, rather than constituting a single benchmark suite. This design allows us to systematically probe how different structural and algorithmic characteristics influence testability across diverse scenarios. We complement this initial set with the widely used *MQTBench* [49] and *QuRaTest* [57] benchmarks to ensure broader coverage, facilitate comparison with prior work, and validate that our observations generalize beyond curated examples when addressing the remaining research questions.

Quantum Software Testability Metric. Testability is measured by systematically introducing controlled perturbations into the state vector and quantifying the resulting output deviation, as illustrated in Figure 2:

- Each circuit is cut at all gate boundaries, creating multiple evaluation points for fault injection.
- Perturbations are introduced as single-qubit or multi-qubit state deviations with fidelity ranging from 0.0 to 1.0 in steps of 0.1.
- For each perturbation, the output state is compared to the original using Hellinger distance, capturing the sensitivity of the circuit to small faults.
- The distribution of Hellinger distances across all cuts and fidelities is used to compute the circuit’s overall testability score.

Quantum Squeeziness Metric. Quantum Squeeziness captures the degree to which information about injected disturbances is preserved or attenuated through the circuit. The circuit is modelled as an information channel, as shown in Figure 3:

- **Input:** Fidelity distance of the injected disturbance.
- **Output:** Hellinger distance between perturbed and original outputs.

- **Computation:** Squeeziness is quantified as the entropy difference between input and output distributions.

Using the Blahut–Arimoto algorithm, we estimated channel capacity and computed mutual information between the ideal reference channel and the channel induced by fault propagation.

FastSqueeziness. We enhanced the computation of Quantum Squeeziness to achieve higher speed while preserving accuracy. First, we evaluated all combinations of ten fidelity ranges between 0 and 1, with a step of 0.1. Second, we sequentially removed circuit cuts and assessed the resulting impact on both accuracy and computational cost. The combination of these optimisations led to a faster variant of the metric, referred to as *Fast Squeeziness*. *FastSqueeziness* reduces computation by selecting representative fidelity points, removing low-impact cuts, and flattening dynamic circuits. Finally, we applied *Fast Squeeziness* to the larger circuit benchmark from Ye et al. [57]. Since some circuits in this benchmark were dynamic, we systematically converted each dynamic circuit into all possible execution paths to enable state-vector analysis.

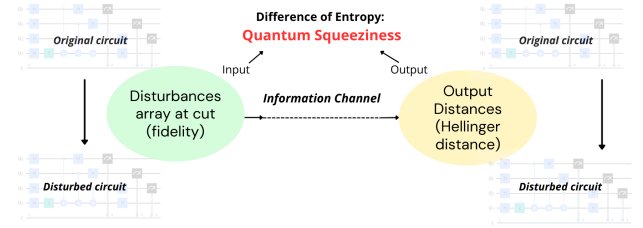


Figure 3: Pipeline to compute Quantum Squeeziness.

5.4 Fault Injection

Initially, modified state vectors were generated by repeated random sampling until the desired fidelity range was reached. For 4-qubit systems and fidelities above 0.6, this approach became impractical, often requiring millions of attempts. After testing different strategies, we found that separating the fidelity regimes was optimal: for fidelities above 0.9, the modified state is constructed geometrically as a controlled combination of the original state and a randomly generated orthogonal component, ensuring precise fidelity by construction. For lower fidelities, the current state is slightly perturbed using a near-identity unitary derived from a small random Hermitian matrix, allowing smooth and controlled exploration of nearby states. Plateau detection and occasional random resets prevent stagnation, ensuring rapid convergence. In most cases, the target fidelity is achieved in the first attempt. This fault-injection approach is not limited to classical simulation. Analogous perturbations could be implemented on quantum hardware via controlled unitary operations or noise engineering techniques, enabling experimental exploration of such faults in real quantum computations.

Why perturbations instead of input variation. Input variation conflates algorithmic behaviour with fault propagation. By injecting controlled perturbations directly into the state evolution, we isolate fault propagation and obtain a testability measure independent of

the function computed by the circuit. The Shannon-Squeeziness baseline (Section 5.2) empirically illustrates the limitations of the input-variation approach.

5.5 Handling Dynamic Circuits

The QuraTest benchmark [57], used in our larger experiment, includes dynamic circuits containing mid-circuit measurements implemented via conditional operations. These circuits are incompatible with state-vector simulation. To address this, we implemented a function that systematically generates all deterministic execution paths for each dynamic circuit. Each conditional operation forces the measured qubit into either 0 or 1 and applies the corresponding sequence of gates, yielding a set of deterministic sub-circuits suitable for state-vector analysis.

5.6 Mutation-Based Validation and Mutation Score

To validate squeeziness, we used Muskit [37] to generate mutants of the selected circuits. Only non-equivalent mutants within 5% of the original circuit's squeeziness were retained to ensure comparability. For dynamic circuits with highly similar branches, we subsampled 10% of mutants per branch to avoid overrepresentation.

The mutation score is defined as the percentage of mutants detected by the test oracle. For each mutant, its output distribution is compared against the oracle using a chi-square test with a significance threshold of 5% ($p < 0.05$). A mutant is classified as detected if more than 10% of its test inputs exhibit a statistically significant deviation. The mutation score is obtained by dividing the number of detected mutants by the total number of mutants. Figure 4 shows the validation pipeline described here to answer RQ3.

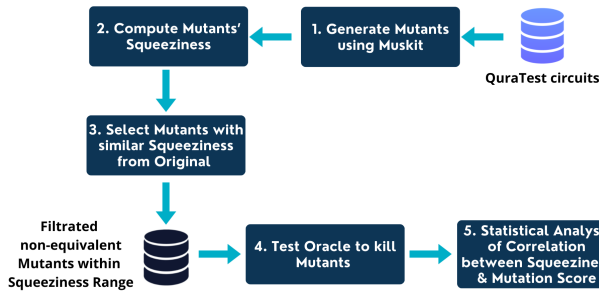


Figure 4: Validation pipeline for Quantum Squeeziness. Mutants are generated using Muskit, filtered to remain within 5% of the original circuit's squeeziness, and tested using a fidelity-based oracle. Correlations between squeeziness and mutation scores are computed to assess the metric's predictive power.

5.7 Experimental Environment and Reproducibility

All experiments were conducted using Qiskit version 1.3.2 and Python 3.9.6 to ensure optimal compatibility. Known algorithms

were obtained from the MQT benchmark [49] as QASM files, while constant and identity circuits were implemented directly in QASM. For larger-scale experiments, we used circuits from Ye et al. [57]. The fully documented replication package is available at [Zenodo](#). All random processes were executed with fixed seeds to ensure reproducibility.

6 Results

In this section, we present our results, organised around each research questions.

6.1 RQ1: Definitions – Quantum software testability and Quantum Squeeziness

Classical notions of software testability rely on deterministic fault propagation and observable outputs. In quantum systems, these assumptions fail due to superposition, entanglement, and measurement-induced collapse. Faults do not propagate deterministically, and measurement yields only probabilistic information. Classical testability metrics, therefore, cannot reliably capture quantum circuit behavior. Figure 5 represents two extreme examples violating our requirements.

We define two complementary concepts:

- (1) **Quantum Testability:** the ability of a quantum circuit to propagate injected faults to outputs so that they can be observed. This captures the intuition from Section 4: circuits that preserve deviations in the output are highly testable, while those that mask perturbations are not.
- (2) **Quantum Squeeziness:** an information-theoretic metric quantifying the loss of information between a faulty input and the resulting output distribution. Squeeziness operationalizes the testability spectrum from Section 4, measuring how observable outputs are affected by faults.

Constant circuits suppress disturbances and show low testability with high squeeziness, whereas Identity circuits propagate all deviations and show high testability with low squeeziness. These definitions satisfy the validation requirements 1 and 2 derived from the intuition and testability spectrum in Section 4.2.

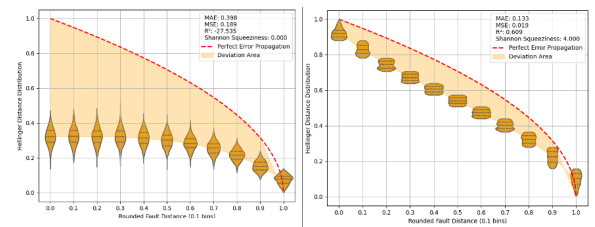


Figure 5: A Naive definition of Squeeziness violates our requirements: On the left the equally balanced circuit has high FEP but a Shanon Squeeziness of 0 (domain and range have the same entropy), and on the right the Identity circuit has very low FEP but a high Shanon Squeeziness of 4 (only one possible output leads to a high difference of entropy).

RQ1: Quantum Testability and Squeeziness

Quantum testability is the expected observable divergence caused by controlled perturbations, capturing how strongly faults propagate to the output. Quantum Squeeziness measures the entropy loss between injected disturbances and resulting deviations, quantifying information compression during computation.

Circuits that suppress perturbations exhibit low testability and high squeeziness, whereas circuits that preserve or amplify them exhibit the opposite. This formalisation avoids naive classical adaptations by accounting for superposition, measurement collapse, and probabilistic observability in quantum programs.

6.2 RQ2: Metrics – Quantifying Testability and Squeeziness

Baseline experiments. Extreme cases illustrate the spectrum of error propagation (Figure 7). Constant circuits show minimal propagation, indicating low testability, while Identity circuits propagate faults strongly, indicating high testability. Here, R^2 refers to the coefficient of determination measuring the distance between the ideal fault propagation predicted by Eq. (3) and the empirical observations, and serves as an empirical metric for assessing fault propagation. This is consistent with requirements 1 and 2 set in Sec. 4.2.

Quantum Squeeziness metric. Circuits that propagate faults efficiently have low squeeziness, while circuits that suppress faults have high squeeziness. The metric correlates inversely with testability ($\text{Spearman } \rho = -0.85$) as indicated in Figure 6 and Table 2. This is consistent with requirement 1 set in Sec. 4.2.

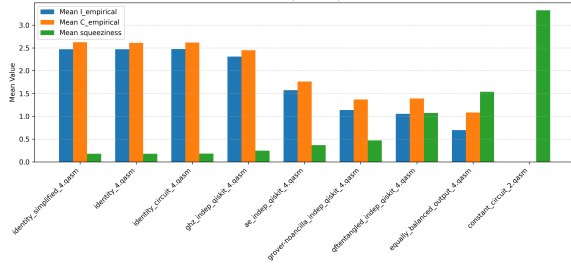


Figure 6: Quantum Squeeziness across circuits. Low squeeziness corresponds to high fault propagation (high testability).

FastSqueeziness optimisation. Across the full dataset (50,000+ mutants), the average runtime per circuit decreases from 3200 s to 189 s, corresponding to a 16.9 $\times$ reduction. Despite this acceleration, result quality is preserved: FastSqueeziness maintains high accuracy (95% on average, with a standard deviation of 9.3%), and mean squeeziness remains comparable to the regular method (regular: 0.99, fast: 1.02). These results demonstrate that FastSqueeziness enables scalable evaluation without sacrificing accuracy.

Detailed circuit results. Table 2 lists Squeeziness and R^2 values, ordered from low to high squeeziness. Identity circuits have low squeeziness and strong positive correlation with testability, whereas

Constant and equally balanced circuits have high squeeziness and weaker or negative correlation.

Table 2: Quantum Squeeziness and R^2 across circuits (low to high Squeeziness).

Circuit	Squeeziness	R^2
identity_simplified_4	0.174	0.605
grover-noancilla_4	0.176	0.912
identity_4	0.184	0.611
identity_circuit_4	0.181	0.608
ae_4	0.339	0.171
ghz_4	0.244	0.453
qftentangled_4	1.075	-9.471
equally_balanced_output_4	1.536	-27.739
constant_circuit_2	3.322	-1.16e+30

RQ2: Metric Properties and Efficiency

Quantum Squeeziness—measuring entropy loss between injected faults and observed deviations—is strongly and inversely correlated with testability (Spearman $\rho = -0.85$). Identity circuits show minimal squeeziness and maximal testability, while constant-output circuits exhibit the opposite, confirming theoretical expectations.

FastSqueeziness reduces computation via cut and perturbation sampling, achieving a 16.9 $\times$ speedup while preserving 95% ranking accuracy, enabling scalable evaluation without exhaustive state perturbation.

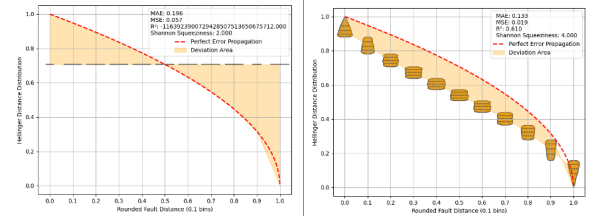


Figure 7: Extreme error propagation: Constant circuit suppresses faults (left), Identity circuit propagates faults strongly (right).

6.3 RQ3: Implications of Testability in Testing

Figure 8 illustrates the relationship between Quantum Squeeziness and mutation score. Across more than 50,000 mutants, evaluated over three months on three parallel nodes, **circuits with higher squeeziness consistently achieve lower mutation scores**. This trend is strongly supported by Pearson correlation ($r = -0.563$, $p = 1.54 \times 10^{-04}$), indicating that fault detection becomes significantly harder as squeeziness increases.

These results answer RQ3: *squeeziness circuits are demonstrably harder to test*, as high-squeeziness circuits suppress fault propagation, whereas low-squeeziness circuits, such as identity-like structures, expose faults more readily.

To further answer RQ3, **Quantum Squeeziness can guide circuit selection for testing**, enabling the identification of circuits where testing is most effective and highlighting those that may require stronger oracles. Overall, *high squeeziness predicts low fault detectability, while low squeeziness identifies circuits with inherently higher testability*, which aligns with its classical counterpart [9].

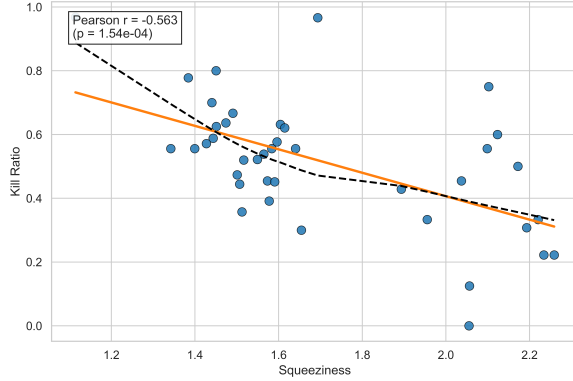


Figure 8: Correlation between Quantum Squeeziness and mutation score. High squeeziness correlates with low mutation score, indicating reduced testability.

Sensitivity to the squeeziness filter. To verify that the observed correlation is not an artefact of the 5% squeeziness filter (Section 5.6), we repeated the entire RQ3 pipeline while varying the filter threshold from 1% up to no filter at all, re-running the same mutant-selection, oracle, and correlation steps used for the headline result at each setting. Table 3 reports, for each threshold, the number of circuits retained and the resulting Pearson and Spearman correlations between squeeziness and mutation score. The negative correlation remains stable and statistically significant across all thresholds, confirming that the filter serves its intended purpose of comparing mutants within the same testability neighbourhood rather than driving the conclusion. Correlations at every threshold are computed with the identical procedure as the headline result, including its point-sampling step. The driver reproducing this table is included in the replication package (sensitivity_table.py).

Table 3: Sensitivity of the squeeziness–mutation-score correlation to the mutant filter threshold. The -0.563 Pearson value at the 5% setting is the headline RQ3 correlation.

Threshold	#Circuits	Pearson r	p	Spearman ρ	p
1%	278	-0.361	$2.2e-02$	-0.228	$1.6e-01$
2.5%	284	-0.044	$7.9e-01$	-0.147	$3.7e-01$
5% (used)	288	-0.563	$1.5e-04$	-0.539	$3.3e-04$
10%	288	-0.560	$1.7e-04$	-0.530	$4.3e-04$
20%	288	-0.506	$8.7e-04$	-0.470	$2.2e-03$
No filter	288	-0.248	$1.2e-01$	-0.161	$3.2e-01$

On the oracle thresholds. The mutation oracle inherits two thresholds from Muskit’s defaults: the chi-square significance level ($p < 0.05$) and the detection rule (a mutant is detected if more than 10%

of its test inputs deviate significantly). We adopt these default settings and rely on the empirical justification provided in the original Muskit study [37], which validated them across a range of quantum circuits; the per-input chi-square results that feed the oracle are available in the replication package for readers who wish to apply a different decision rule.

RQ3: Implications for Quantum Testing

Quantum Squeeziness is negatively correlated with mutation score ($r = -0.563$): circuits with higher information compression are harder to test, while low-squeeziness circuits expose faults more reliably.

Squeeziness therefore predicts test effectiveness and can guide circuit selection, test prioritisation, and the use of stronger statistical or oracle-based techniques where needed.

7 Discussion

In this Section, we discuss the results presented earlier.

7.1 Stability of Squeeziness Across the Circuit

Our experiments indicate that the location of the injected fault in a noiseless quantum circuit has minimal impact on its propagation. We attribute this to the inherent reversibility of ideal quantum operations. Importantly, we do not expect hardware noise to invalidate the approach. Rather, we expect squeeziness to vary under noise: noise breaks the reversibility of the ideal evolution and alters fault-propagation behaviour, so that squeeziness becomes location- and noise-dependent. This is itself an opportunity—analysing the variation of squeeziness across cuts could expose specific sources of hardware noise, such as faulty gates or qubit decoherence, and could inform noise-mitigation mechanisms. Future work should evaluate how noise, entanglement, and circuit depth affect squeeziness stability and whether localized deviations can be systematically used for hardware diagnostics.

7.2 Squeeziness of Mutants

We observed that squeeziness responds differently to mutations depending on the circuit. While mutant generation is independent of predefined squeeziness (see Section 5 for details on the mutation process), some circuits produce only a few mutants that happen to fall within a given squeeziness range of the original circuit, whereas others allow many more. This variability suggests that certain circuits are more structurally robust or “squeeziness-stable” than others, possibly due to differences in entanglement structure, gate composition, or circuit depth. Further investigation is warranted to understand which circuit properties influence the distribution of mutants within a squeeziness threshold and how this insight could guide both mutation-based testing and quantum software design.

7.3 Error Propagation Behaviors of Circuits

Our results show that quantum circuits exhibit distinct error propagation behaviours, ranging from fault masking to effective fault propagation. These behaviours determine testability: circuits that propagate perturbations to the output are more testable, while

those that suppress them exhibit lower testability and higher Quantum Squeeziness. This allows us to link testability to structural aspects of quantum circuit design. Future work should identify sub-circuits or patterns that influence squeeziness and analyze how design choices—such as gate selection, qubit connectivity, and entanglement—affect the testability of larger circuits. As quantum software grows in complexity, testing challenges will increase, making design-for-testability considerations potentially even more critical than in classical software [52].

7.4 Implications for Quantum Software Engineering

Overall, our findings suggest that squeeziness is a promising metric for guiding quantum software testing and design. It can serve as a fast and efficient method for selecting testable mutants, assessing error propagation tendencies, and identifying structurally sensitive sub-circuits. Integrating squeeziness into quantum software development pipelines could facilitate the early detection of design weaknesses and improve overall software robustness, especially as circuits scale and hardware noise becomes a more significant factor.

8 Threats to Validity

In this section, we discuss the potential threats to the validity of our study.

8.1 Internal Threats

The perturbation model used throughout our experiments includes a partial trace appears as an internal implementation detail. Although this introduces a fixed approximation, it does not affect the relative comparison of squeeziness across circuits. The partial-trace mechanism was chosen for its practicality in state-vector simulations, enabling precise control over fault injection and error propagation. While more optimized implementations—such as improved information-channel mechanisms—may enhance computational efficiency, the current design remains sufficient for reliable testing and benchmarking. These limitations therefore impact absolute accuracy rather than internal consistency.

8.2 External Threats

Quantum Squeeziness currently relies on simulator-level state-vector access, which cannot be reproduced on physical quantum hardware. Real devices provide only measurement outcomes, and noise affects both state evolution and readout. This limits direct applicability and poses a major threat to external validity.

A central challenge is defining or approximating the “final state vector” on hardware. Direct reconstruction is infeasible, but weak or gentle measurements, mid-circuit operations, and classical-shadow techniques offer promising alternatives. Parallel work has developed an efficient, scalable method for comparing quantum states [4, 32], which we plan to adapt for this purpose. In parallel work, we have developed a classical-shadow-based approach for scalable, hardware-compatible state comparison and live noise fingerprinting [4, 32]. It leverages the logarithmic sample complexity of classical shadows [23] to compare large quantum states from measurement outcomes alone, removing the dependence on full

state-vector access, and provides a concrete path toward instantiating the squeeziness pipeline in a measurement-based setting. Developing a practical hardware-compatible formulation of squeeziness remains an important direction for future work.

9 Future Work

In this study, we introduced Quantum Squeeziness, a novel metric for quantum software testability, and demonstrated one application: selecting mutants within a defined squeeziness range. Future work will explore additional applications, including integrating squeeziness into differential testing based on fuzzing and using it as a fitness function in automated quantum circuit generation or evolutionary algorithms. We are currently developing entropy-based fuzzing approaches that use squeeziness as a fitness function to guide input and perturbation generation, with promising preliminary results.

We also plan to investigate hardware applicability by analyzing squeeziness in noisy environments, examining how variations across cuts could reveal noise or hardware errors. Furthermore, we aim to extend the notion of squeeziness to guide and optimize the design of circuits in hybrid quantum-classical architectures, potentially improving performance and testability in variational and other hybrid algorithms.

10 Conclusion

This work introduced the first formalisation of testability for quantum programs, establishing a foundation for analyzing how faults propagate to observable failures. By defining Quantum Squeeziness and proposing efficient computation methods, we showed that testability can be quantified in a quantum context and correlates strongly with fault detectability. Our experiments demonstrated that Quantum Squeeziness provides accurate and computationally efficient insights, with FastSqueeziness offering substantial performance gains while maintaining high fidelity.

Looking forward, we envision broader applications of squeeziness: guiding automated quantum circuit generation, serving as a fitness function in evolutionary algorithms, and informing design guidelines by identifying sub-circuits that contribute most to low testability. Evaluating squeeziness under noise could enable efficient error detection and fault localisation on real hardware. These directions highlight the potential for integrating testability analysis into quantum software development, with implications for testing, debugging, error mitigation, and circuit design.

Data Availability Statement The replication package supporting this paper is available on [Zenodo](#) [5].

Acknowledgments. Avner Bensoussan and Mohammad Reza Mousavi have been partially supported by the UKRI Trustworthy Autonomous Systems Node in Verifiability, Grant Award Reference EP/V026801/2, EPSRC project on Verified Simulation for Large Quantum Systems (VSL-Q), grant reference EP/Y005244/1 and the EPSRC project on Robust and Reliable Quantum Computing (RoarQ), Investigation 009 Model-based monitoring and calibration of quantum computations (ModeMCQ), grant reference EP/W032635/1.

References

- [1] Rui Abreu, João Paulo Fernandes, Luis Llana, and Guilherme Tavares. 2022. Metamorphic Testing of Oracle Quantum Programs. In *2022 IEEE/ACM 3rd International Workshop on Quantum Software Engineering (Q-SE)*. 16–23. doi:10.1145/3528230.3529189
- [2] Kelly Androustopoulos, David Clark, Haitao Dan, Robert M. Hierons, and Mark Harman. 2014. An analysis of the relationship between conditional entropy and failed error propagation in software testing. In *Proceedings of the 36th International Conference on Software Engineering* (New York, NY, USA, 2014-05-31) (ICSE 2014). Association for Computing Machinery, 573–583. doi:10.1145/2568225.2568314
- [3] Charles Bennett. 1998. Quantum Information Theory. *IEEE Transactions on Information Theory* (1998). https://www.academia.edu/3729794/Quantum_Information_Theory
- [4] Avner Bensoussan, Elena Chachkarova, Karine Even-Mendoza, Sophie Fortz, and Vasileios Klimis. 2025. Toward Live Noise Fingerprinting in Quantum Software Engineering. arXiv:2512.18667 [quant-ph] <https://arxiv.org/abs/2512.18667>
- [5] Avner Bensoussan, Hector Menendez, and Mohammad Reza Mousavi. 2026. Artifact of Quantum Squeeziness: Information-Theoretic Foundations of Quantum Software Testability. doi:10.5281/zenodo.21177819
- [6] Robert V. Binder. 1994. Design for testability in object-oriented systems. *Commun. ACM* 37, 9 (1994), 87–101. doi:10.1145/182987.184077
- [7] L. C. Briand, Y. Labiche, and H. Sun. 2003. Investigating the use of analysis contracts to improve the testability of object-oriented code. *Software: Practice and Experience* 33, 7 (2003), 637–672. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/spe.520>. doi:10.1002/spe.520
- [8] José Campos and André Souto. 2021. Q Bugs: A Collection of Reproducible Bugs in Quantum Algorithms and a Supporting Infrastructure to Enable Controlled Quantum Software Testing and Debugging Experiments. arXiv:2103.16968 [cs]. <http://arxiv.org/abs/2103.16968>
- [9] David Clark and Robert M. Hierons. 2012. Squeeziness: An information theoretic measure for avoiding fault masking. *Inform. Process. Lett.* 112, 8 (2012), 335–340. doi:10.1016/j.ipl.2012.01.004
- [10] David Deutsch. 1985. Quantum theory, the Church–Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 400, 1818 (1985), 97–117. doi:10.1098/rspa.1985.0070
- [11] F.J.O. Dias. 1975. Fault Masking in Combinational Logic Circuits. *IEEE Trans. Comput.* C-24, 5 (1975), 476–482. Conference Name: IEEE Transactions on Computers. doi:10.1109/T-C.1975.224249
- [12] Frank Arute et al. 2019. Quantum supremacy using a programmable superconducting processor. *Nature* 574, 7779 (oct 2019), 505–510. doi:10.1038/s41586-019-1666-5
- [13] Richard P. Feynman. 1982. Simulating physics with computers. *International Journal of Theoretical Physics* 21, 6 (1982), 467–488. doi:10.1007/BF02650179
- [14] Daniel Fortunato, José Campos, and Rui Abreu. 2022. Mutation testing of quantum programs written in QISKit. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (New York, NY, USA, 2022-10-19) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 358–359. doi:10.1145/3510454.3528649
- [15] Daniel Fortunato, Luis Jiménez-Navajas, José Campos, and Rui Abreu. 2024. Verification and Validation of Quantum Software. In *Quantum Software: Aspects of Theory and System Design*, Iakov Exman, Ricardo Pérez-Castillo, Mario Piattini, and Michael Felderer (Eds.). Springer Nature Switzerland, Cham, 93–123. doi:10.1007/978-3-031-64136-7_5
- [16] Antonio García de la Barrera, Ignacio García-Rodríguez de Guzmán, Macario Polo, and Mario Piattini. 2023. Quantum software testing: State of the art. *Journal of Software: Evolution and Process* 35, 4 (2023), e2419. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.2419>. doi:10.1002/smr.2419
- [17] Vahid Gharousi, Michael Felderer, and Feyza Nur Kılıçaslan. 2019. A survey on software testability. *Information and Software Technology* 108 (April 2019), 35–64. doi:10.1016/j.infsof.2018.12.003
- [18] Sukhpal Singh Gill, Adarsh Kumar, Harvinder Singh, Manmeet Singh, Kamalpreet Kaur, Muhammad Usman, and Rajkumar Buyya. 2022. Quantum computing: A taxonomy, systematic review and future directions. *Software: Practice and Experience* 52, 1 (2022), 66–114. doi:10.1002/spe.3039
- [19] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. doi:10.1145/237814.237866
- [20] E. Hellinger. 1909. Neue Begründung der Theorie quadratischer Formen von unendlichvielen Veränderlichen. *Journal für die reine und angewandte Mathematik* 136 (1909), 210–271. <http://eudml.org/doc/149313>
- [21] Julian Hirt, Thomas Nordhausen, Thomas Fuerst, Hannah Ewald, and Christian Appenzeller-Herzog. 2024. Guidance on terminology, application, and reporting of citation searching: the TARCIS statement. *BMJ* 385 (May 2024), e078384. Publisher: British Medical Journal Publishing Group Section: Research Methods & Reporting. doi:10.1136/bmj-2023-078384
- [22] Shahin Honarvar, Mohammad Reza Mousavi, and Rajagopal Nagarajan. 2020. Property-based Testing of Quantum Programs in Q#. *Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops* (2020). <https://api.semanticscholar.org/CorpusID:219977843>
- [23] Hsin-Yuan Huang, Richard Kueng, and John Preskill. 2020. Predicting Many Properties of a Quantum System from Very Few Measurements. *Nature Physics* 16, 10 (Oct. 2020), 1050–1057. arXiv:2002.08953 [quant-ph]. doi:10.1038/s41567-020-0932-7
- [24] Yipeng Huang and Margaret Martonosi. 2019. Statistical Assertions for Validating Patterns and Finding Bugs in Quantum Programs. In *Proceedings of the 46th International Symposium on Computer Architecture*. 541–553. arXiv:1905.09721 [quant-ph]. doi:10.1145/3307650.3322213
- [25] Alfredo Ibañez, Robert M. Hierons, and Manuel Núñez. 2019. Using Squeeziness to test component-based systems defined as Finite State Machines. *Information and Software Technology* 112 (2019), 132–147. doi:10.1016/j.infsof.2019.04.012
- [26] IEEE Standards Association. 1990. IEEE Standards Association. <https://standards.ieee.org/ieee/610.12/855/>
- [27] ISO/IEC. 2024. ISO/IEC 25002:2024 – Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality model overview and usage. <https://www.iso.org/standard/78175.html>
- [28] Niraj K. Jha. 1996. Fault-tolerant computer system design. *IEEE Parallel & Distributed Technology: Systems & Applications* 4, 4 (1996), 84–84. Conference Name: IEEE Parallel & Distributed Technology: Systems & Applications. doi:10.1109/MPDT.1996.7102341
- [29] Barry W. Johnson. 1989. *Design and Analysis of Fault-tolerant Digital Systems*. Addison-Wesley Publishing Company.
- [30] Richard Jozsa. 1994. Fidelity for Mixed Quantum States. *Journal of Modern Optics* 41, 12 (Dec. 1994), 2315–2323. doi:10.1080/09500349414552171
- [31] Chan Gu Kang, Joonghoon Lee, and Hakjoo Oh. 2024. Statistical Testing of Quantum Programs via Fixed-Point Amplitude Amplification. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 276 (Oct. 2024), 25 pages. doi:10.1145/3689716
- [32] Vasileios Klimis, Avner Bensoussan, Elena Chachkarova, Karine Even-Mendoza, Sophie Fortz, and Connor Lenihan. 2025. Shaking Up Quantum Simulators with Fuzzing and Rigour. In *Proceedings of the ACM on Programming Languages*, Vol. 9. ACM, 1400–1428. doi:10.1145/3763100
- [33] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2023. QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation. *ACM Transactions on Quantum Computing* 4, 2 (2023), 10:1–10:26. doi:10.1145/3550488
- [34] Gushu Li, Li Zhou, Nengkun Yu, Yufei Ding, Mingsheng Ying, and Yuan Xie. 2020. Proq: Projection-based Runtime Assertions for Debugging on a Quantum Computer. arXiv:1911.12855 [quant-ph]. <http://arxiv.org/abs/1911.12855>
- [35] Zhimin Li, Harshitha Menon, Kathryn Mohror, Peer-Timo Bremer, Yarden Livnat, and Valerio Pascucci. 2021. Understanding a program’s resiliency through error propagation. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM, Virtual Event Republic of Korea, 362–373. doi:10.1145/3437801.3441589
- [36] Junjie Luo, Pengzhan Zhao, Zhongtao Miao, Shuhan Lan, and Jianjun Zhao. 2022. A Comprehensive Study of Bug Fixes in Quantum Programs. <https://www.computer.org/csdl/proceedings-article/saner/2022/378600b239/1FbT6n3hGaA>
- [37] Eñaut Mendiluze, Shaikat Ali, Paolo Arcaini, and Tao Yue. 2021. Muskiet: A Mutation Analysis Tool for Quantum Software Testing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2021-11). IEEE, Los Alamitos, CA, USA, 1266–1270. ISSN: 2643-1572. doi:10.1109/ASE51524.2021.9678563
- [38] Sara Ayman Metwalli and Rodney Van Meter. 2024. Testing and Debugging Quantum Circuits. *IEEE Transactions on Quantum Engineering* (2024), 1–15. doi:10.1109/TQE.2024.3374879
- [39] Sara Ayman Metwalli and Rodney Van Meter. 2022. A Tool For Debugging Quantum Circuits. <https://www.computer.org/csdl/proceedings-article/qce/2022/911300a624/1LvLZRLty80>
- [40] K.W. Miller, L.J. Morell, R.E. Noonan, S.K. Park, D.M. Nicol, B.W. Murrill, and M. Voas. 1992. Estimating the probability of failure when testing reveals no failures. *IEEE Transactions on Software Engineering* 18, 1 (1992), 33–43. doi:10.1109/32.120314
- [41] Andriy Miranskyy and Lei Zhang. 2019. On Testing Quantum Programs. In *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)* (2019-05). 57–60. arXiv:1812.09261 [quant-ph] doi:10.1109/ICSE-NIER.2019.00023
- [42] Michael A. Nielsen and Isaac L. Chuang. 2010. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press. ISBN: 9780511976667 Publisher: Cambridge University Press. doi:10.1017/CBO9780511976667
- [43] Noah H. Oldfield, Christoph Laaber, Tao Yue, and Shaikat Ali. 2024. Faster and Better Quantum Software Testing through Specification Reduction and Projective Measurements. arXiv:2405.15450 [quant-ph] <http://arxiv.org/abs/2405.15450>
- [44] Matteo Paltenghi and Michael Pradel. 2022. Bugs in Quantum computing platforms: an empirical study. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 86 (April 2022), 27 pages. doi:10.1145/3527330
- [45] Matteo Paltenghi and Michael Pradel. 2023. MorphQ: Metamorphic Testing of the Qiskit Quantum Computing Platform. In *IEEE/ACM 45th International*

- Conference on Software Engineering (ICSE) (ICSE '23)*. IEEE Press, Melbourne, Victoria, Australia, 2413–2424. doi:10.1109/ICSE48619.2023.00202
- [46] Krishna Patel, Robert M. Hierons, and David Clark. 2022. An information theoretic notion of software testability. *Information and Software Technology* 143 (2022), 106759. doi:10.1016/j.infsof.2021.106759
- [47] Gabriel Joseph Pontolillo, Mohammad Reza Mousavi, and Marek Grzesiuk. 2025. QuCheck: A Property-based Testing Framework for Quantum Programs in Qiskit. arXiv preprint. arXiv:2503.22641 [quant-ph] doi:10.48550/arXiv.2503.22641
- [48] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (Aug. 2018), 79. arXiv:1801.00862 [cond-mat, physics:quant-ph]. doi:10.22331/q-2018-08-06-79
- [49] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing. *Quantum* (2023). MQT Bench is available at <https://www.cda.cit.tum.de/mqtbench/>. arXiv:2204.13719 [quant-ph] doi:10.22331/q-2023-07-20-1062
- [50] C. E. Shannon. 1948. A Mathematical Theory of Communication. *Bell System Technical Journal* 27, 3 (1948), 379–423. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/j.1538-7305.1948.tb01338.x>. doi:10.1002/j.1538-7305.1948.tb01338.x
- [51] Peter W. Shor. 1997. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM J. Comput.* 26, 5 (Oct. 1997), 1484–1509. doi:10.1137/S0097539795293172
- [52] J.M. Voas and K.W. Miller. 1995. Software testability: the new verification. *IEEE Software* 12, 3 (1995), 17–28. Conference Name: IEEE Software. doi:10.1109/52.382180
- [53] Xinyi Wang, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2022. QuSBT: search-based testing of quantum programs. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 173–177. doi:10.1145/3510454.3516839
- [54] Xinyi Wang, Tongxuan Yu, Paolo Arcaini, Tao Yue, and Shaukat Ali. 2022. Mutation-based test generation for quantum programs with multi-objective search. In *Proceedings of the Genetic and Evolutionary Computation Conference (Boston, Massachusetts) (GECCO '22)*. Association for Computing Machinery, New York, NY, USA, 1345–1353. doi:10.1145/3512290.3528869
- [55] Mark M. Wilde. 2017. *Quantum Information Theory* (2nd ed.). Cambridge University Press. doi:10.1017/9781316809976
- [56] Martin R. Woodward and Zuhoor A. Al-Khanjari. 2000. *Testability, fault size and the domain-to-range ratio: An eternal triangle*. Association for Computing Machinery, New York, NY, USA, 168–172 pages. doi:10.1145/347324.349016
- [57] Jiaming Ye, Shangzhou Xia, Fuyuan Zhang, Paolo Arcaini, Lei Ma, Jianjun Zhao, and Fuyuki Ishikawa. 2024. QuraTest: Integrating Quantum Specific Features in Quantum Program Testing. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (Echternach, Luxembourg) (ASE '23)*. IEEE Press, Los Alamitos, CA, USA, 1149–1161. doi:10.1109/ASE56229.2023.00196
- [58] Masoud Zamani, Navid Farazmand, and Mehdi B. Tahoori. 2011. Fault Masking and Diagnosis in Reversible Circuits. In *2011 Sixteenth IEEE European Test Symposium* (2011-05). 69–74. ISSN: 1558-1780. doi:10.1109/ETS.2011.19
- [59] Pengzhan Zhao, Jianjun Zhao, and Lei Ma. 2021. Identifying Bug Patterns in Quantum Programs. 16–21 pages. doi:10.1109/Q-SE52541.2021.00011
- [60] Pengzhan Zhao, Jianjun Zhao, Zhongtao Miao, and Shuhan Lan. 2021. Bugs4Q: A Benchmark of Real Bugs for Quantum Programs. arXiv:2108.09744 [cs]. <http://arxiv.org/abs/2108.09744>

Received 2026-03-26; accepted 2026-06-18